Title: Credit Card Fraud Detection under Extreme Class Imbalance: Resampling, Cost-Sensitive Learning and Ensembles

First name	Last Name	IIT Email	Team Leader
			x

1. Introduction

Credit card fraud costs banks and merchants a lot of money every year. The only practical way to catch it is to have a model score each transaction as it comes in and flag the ones that look suspicious. What makes this hard is how rare fraud is. In the dataset we picked, only about 0.17% of the transactions are fraud. A model that just says "normal" every single time would be right 99.83% of the time and still catch nothing.

The two kinds of mistakes also do not cost the same. If we miss a fraud, somebody loses money. If we flag a normal purchase, a customer gets their card blocked and an analyst has to spend time checking it. A good detector needs to find most of the fraud without raising so many false alarms that nobody can use it.

We chose this project because it pulls together a lot of what we covered in class: imbalanced data, why accuracy can be misleading, resampling, class weights, ensembles, and evaluation with precision, recall and PR-AUC. Instead of just running a long list of algorithms, we want to find out which way of handling the imbalance actually works best on this data, and what each option costs us.

2. Data Sets

We will use the Credit Card Fraud Detection dataset from Kaggle, published by the Machine Learning Group at ULB: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud. It contains credit card transactions made by European cardholders over two days in September 2013.

The data has 284,807 transactions and 31 columns. Only 492 of the transactions are fraud (0.172%), and the other 284,315 are normal. As far as we know there are no missing values, but we will check that, and we will look for duplicate rows too.

The variables are:

- Time: the number of seconds since the first transaction in the data (numeric).
- V1 to V28: 28 numeric features that came out of a PCA transformation of the original attributes, which are confidential. Because of that we cannot tell what each one means.
- Amount: the transaction amount (numeric, with a strong right skew).
- Class: the target variable, where 1 means fraud and 0 means normal.

Our task is binary classification of Class, with fraud as the positive class.

For preprocessing, we will first check for missing values and duplicates. Then we will normalize Time and Amount, since KNN and SVM need scaled features (V1 to V28 are already on similar scales). Naive Bayes needs categorical features, so for that model we will cut the numeric columns into equal-frequency bins, with the bin edges computed on the training data only. We will split the data with stratification so every split keeps the same fraud rate, and all resampling will happen on the training data only, never on validation or test data.

3. Research Problems

All of our problems are binary classification, with fraud as the positive label and normal as the negative label.

RP1. How much does the extreme imbalance hurt ordinary classifiers? We will train KNN, Naive Bayes, a decision tree and a linear model (logistic regression or linear SVM) on the original, untouched training data and show how a very high accuracy can hide a poor fraud recall. These results are our baseline.

RP2. Does rebalancing the data help, and how much precision do we lose for it? We will compare random undersampling of the normal class with random oversampling of the fraud class (and SMOTE if time allows), always on the training set only.

RP3. Can cost-sensitive learning do as well as resampling? We will try class weights with a decision tree, a linear model and a random forest, and also a balanced random forest, which undersamples inside every tree.

RP4. Do ensembles detect fraud better than single models? We will compare bagging (random forest, balanced random forest), boosting (AdaBoost, gradient boosting or XGBoost) and stacking against our best single models.

RP5. Which features matter, and where should we put the decision threshold? First, we will test whether a smaller set of features, picked by mutual information or by random forest importance, keeps the performance. Second, we will use the precision-recall curve on validation data to choose the threshold that fits an assumed cost ratio between a missed fraud and a false alarm.

4. Potential Solutions

The table below lists the experiments we plan to run. Each one answers one of the research problems above. We will code everything in Python with scikit-learn, imbalanced-learn and XGBoost.

ID	Models	Imbalance handling	Question answered
E1	KNN, Naive Bayes (Laplace smoothing, binned features), decision tree, logistic regression / linear SVM	None (original distribution)	RP1: baseline, shows the accuracy trap
E2	Same models as E1	Random undersampling of the normal class to 1:1 and 1:10 (fraud : normal)	RP2: gain in recall vs loss in precision
E3	Same models as E1	Random oversampling to 1:1; SMOTE as an extension	RP2: does copying or synthesizing fraud help, or just overfit?
E4	Decision tree, logistic regression / linear SVM, random forest	Class weights (balanced)	RP3: class weights vs resampling
E5	Random forest, balanced random forest, AdaBoost, gradient boosting / XGBoost	Built in (balanced bagging, scale_pos_weight, sample weights)	RP4: ensembles vs single models
E6	Stacking: tree, Naive Bayes and logistic regression as base models, logistic regression as the meta-model	Weights or undersampling inside each training fold	RP4: does a meta-model add anything?
E7	Best model from E1 to E6	Best handling from above	RP5: feature selection and threshold choice

We picked these methods with the data in mind. KNN needs numeric, normalized features, so we will scale Time and Amount. It is also slow on about 227,000 training rows, so we will mostly run it on undersampled data and tune K using odd values. Naive Bayes needs categorical features, which is why we bin the numeric columns. Since the fraud class is so small, a zero probability is very likely, so Laplace smoothing matters here and we will tune alpha on the validation folds. Decision trees do not need scaling but overfit easily, so we will tune the depth. A kernel SVM will not scale to this many rows, so we will use a linear SVM or train on undersampled data.

Resampling will only be done on the training folds, inside the cross-validation loop. In a training split of about 227,800 rows, undersampling to 1:1 leaves about 788 rows and oversampling to 1:1 gives about 455,000 rows. If we resampled before splitting, copies or synthetic frauds would leak into the validation data and make the scores look better than they are. For the stacking model we will build the meta-model from out-of-fold predictions for the same reason.

5. Evaluations

We will make one stratified 80/20 split. The 80% part (about 227,800 rows and about 394 frauds) is used for training and for tuning with stratified 5-fold cross-validation. The 20% part (about 57,000 rows and about 98 frauds) is a hold-out test set that we use once to report each final model. We chose cross-validation for tuning because there are so few frauds that a single validation split would be unstable. The test set also has few frauds, so we will repeat the whole process with several random seeds and report the mean and standard deviation. As a sanity check we will also try a time-ordered split (train on earlier transactions, test on later ones) to see how much the results change.

Accuracy will only be reported to show that it is misleading. The metrics we will focus on are:

- Recall on the fraud class: how many of the real frauds we catch.
- Precision on the fraud class: how many of the transactions we flag are really fraud.
- F1 and F2 (F-beta with beta = 2, which counts recall more), along with the confusion matrix, specificity and false positive rate.
- PR-AUC, the area under the precision-recall curve, as our main threshold-free score. Its random baseline is the fraud rate (about 0.0017), so we will read the numbers against that. We will report ROC-AUC as well, but only as a secondary number, because the huge normal class makes the false positive rate look small and can make the score look better than it is.

  Every method will use the same folds and the same test set so the comparison is fair. We will compare the resampling methods against the baseline and against each other, at the default threshold and at the threshold chosen on validation data. For the threshold question we will assume an example cost ratio, for instance a missed fraud costing 10 times a false alarm, and work out the expected cost on the test set. That ratio is only an assumption, so we will state it clearly and try other values. We will judge differences between models over the repeated runs and not from a single score.

6. Expected Outcomes

By the end we expect to have a working, reproducible code base, a results table that compares every experiment on recall, precision, F1, F2 and PR-AUC, the precision-recall curves, and a final report with our recommendations.

We also have some guesses that we want to test. These are expectations and not results:

- The baseline models will reach over 99% accuracy but miss a big share of the fraud, which shows accuracy is not a good measure here.
- Undersampling will raise recall but cut precision a lot, because of the many false alarms from the large normal class. Oversampling and class weights should keep precision better, but they may overfit or train slowly.
- Ensembles, especially balanced random forest and boosting, will probably give the best PR-AUC. Stacking might add only a small gain over the best single ensemble.
- A small group of features will carry most of the signal, so feature selection should cut training time without losing much.
- Choosing the threshold from the precision-recall curve will matter about as much as choosing the model.

We will count the project as a success if PR-AUC ends up far above the 0.0017 random baseline, with fraud recall as high as possible at a precision that is still practical, and if we can explain clearly which imbalance strategy works best and why.

There are some limits we should be upfront about. The V1 to V28 features are anonymized PCA components, so we cannot interpret them or build new features from them easily. The data covers only two days, so the results may not hold over a longer period. And the test set has only about 98 frauds, so the scores will carry some noise.